

Dokumentacja Inżynierii Wymagań - System Kontroli Dostępu SCANAR

Cel dokumentu

Celem niniejszego dokumentu jest szczegółowe określenie wymagań funkcjonalnych i нефункциональных dla systemu kontroli dostępu SCANAR, który łączy weryfikację kodem QR z rozpoznawaniem twarzy. Dokument ten ma na celu zapewnienie spójnego i precyzyjnego opisu oczekiwań, które będą stanowiły podstawę do projektowania, implementacji oraz testowania rozwiązania. System SCANAR ma zwiększyć bezpieczeństwo w zakładach przemysłowych poprzez eliminację możliwości przekazywania kart dostępowych między pracownikami.

Historia wersji

Data	Wersja	Opis zmian
22-01-2026	1.0	Oddanie finalnej wersji dokumentu
21-01-2026	0.4	Aktualizacja stosu technologicznego, zweryfikowanie poprawności DIW ze zrealizowanym projektem, zatwierdzenie DIW
10-12-2025	0.3	DIWMVP, projekt architektury, stos technologiczny
26-11-2025	0.2	Format danych wejściowych/wyjściowych, modelowanie systemu, diagram przypadków użycia, diagram sekwencyjny
19-11-2025	0.1	Macierz kompetencji, zestawy pytań, funkcjonalności,

Spis treści

Dokumentacja Inżynierii Wymagań - System Kontroli Dostępu SCANAR.....	1
Cel dokumentu.....	1
Historia wersji.....	1

Spis treści.....	1
Słownik pojęć.....	2
Członkowie zespołu.....	3
Zestaw pytań sformułowanych podczas spotkania z klientem.....	4
Funkcjonalności (wymagania funkcjonalne).....	5
Wymagania niefunkcjonalne.....	9
Ustalony format danych wejściowych.....	9
Modelowanie systemu.....	10
Diagram przypadków użycia.....	12
Diagram sekwencyjny UML.....	14
Pracownik (użytkownik).....	14
Administrator.....	15
Projekt architektury opracowywanego systemu.....	17
DIWMVP - Definicja Minimum Viable Product.....	18
Stos technologiczny.....	20
Informacje.....	22

Słownik pojęć

- **SCANAR** – System Kontroli Dostępu oparty na dwuetapowej weryfikacji (kod QR + rozpoznawanie twarzy)
- **Kod QR** – unikalny kod przypisany do pracownika, generowany przez system, zawierający identyfikator pracownika
- **Weryfikacja dwuetapowa** – proces polegający na skanowaniu kodu QR, a następnie weryfikacji twarzy
- **Rozpoznawanie twarzy** – proces biometryczny porównujący obraz z kamery z wzorcami twarzy zapisanymi w bazie danych
- **Tolerancja rozpoznawania** – parametr określający dokładność dopasowania twarzy (w systemie: 0.6)
- **Panel administracyjny** – interfejs webowy służący do zarządzania pracownikami, kodami QR oraz przeglądania logów dostępu
- **Logi dostępu** – rejestr wszystkich prób dostępu (udanych i nieudanych) z danymi: timestamp, ID pracownika, wynik weryfikacji, typ zdarzenia
- **Typy zdarzeń** – INVALID_QR, FACE_MISMATCH, VERIFICATION_SUCCESS, QR_SCANNED, FACE_SCANNED
- **Baza danych pracowników** – SQLite zawierająca dane: ID, imię, nazwisko, ścieżki do zdjęć twarzy, ID kodu QR, status aktywności
- **Wzorce twarzy** – zestaw zdjęć lub wideo pracownika zapisany w formacie .jpg
- **Status kodu QR** – aktywny/nieaktywny (możliwość dezaktywacji w przypadku utraty)

- **REST API** – interfejs programistyczny umożliwiający komunikację między frontendem a backendem
- **CRUD** – operacje Create, Read, Update, Delete na rekordach pracowników

Członkowie zespołu

Tabela 1: Macierz kompetencji zespołu

Kompetencje	Paweł Kowalcze	Mateusz Gacek	Jan Ogiegło
Programowanie Python (Flask)	Posiada	Posiada	Posiada
Przetwarzanie obrazu (np. OpenCV)	Nie posiada	Nie posiada	Nie posiada
Bazy Danych • SQL • SQLAlchemy	Posiada (podstawy)	Posiada (podstawy)	Posiada (podstawy)
React 19.0.2 + Hooks	Posiada (zaawansowany)	Nie posiada	Nie posiada
DevOps (Docker)	Nie posiada	Posiada (podstawy)	Posiada
Znajomość UML	Posiada	Posiada	Posiada
Tworzenie wizualizacji frontendu (Figma)	Posiada	Posiada	Posiada
Testowanie oprogramowania	Posiada (podstawy)	Posiada (podstawy)	Posiada (podstawy)
Kontrola wersji (Git)	Posiada	Posiada	Posiada
Tworzenie dokumentacji	Posiada	Posiada	Posiada
Code review	Posiada	Posiada	Posiada
Metody Agile/Scrum	Posiada	Posiada	Posiada

Zestaw pytań sformułowanych podczas spotkania z klientem

W tabeli poniżej umieszczono zestaw pytań, które zostały sformułowane w celu uszczegółowienia projektu. Zanotowano odpowiedzi, które pojawiły się w trakcie dyskusji.

Tabela 2: Pytania i odpowiedzi z fazy analitycznej

Pytanie	Odpowiedź	Uwagi
Jak wygląda proces dodawania nowego pracownika do systemu?	Administrator przez panel webowy wprowadza dane osobowe pracownika (imię, nazwisko) oraz przesyła zdjęcia twarz. System automatycznie generuje unikalny kod QR przypisany do ID pracownika	Proces musi być obsługiwany przez dedykowany panel administracyjny.
Co w sytuacji, gdy system błędnie nie rozpozna uprawnionego pracownika?	Taka próba jest logowana jako nieudana. Możliwe jest ponowne przystąpienie do procedury - obowiązuje limit nieudanych prób. Po tym procedura zakłada ręczną weryfikację przez pracownika ochrony, który może otworzyć bramkę po sprawdzeniu dokumentów.	Kluczowe jest raportowanie fałszywych negatywów w celu kalibracji systemu.
Jakie dane mają być zawarte w raportach?	Raporty powinny zawierać statystyki poprawnych i niepoprawnych wejść, identyfikować próby nadużyć (np. użycie kodu QR przez inną osobę) oraz logować czas wejścia	Raporty powinny być generowane w określonych przedziałach czasowych (dziennie).
W jaki sposób pracownik prezentuje kod QR?	Dopuszczalne jest skanowanie kodu z ekranu telefonu komórkowego lub z wydrukowanej przepustki.	Skaner musi być w stanie odczytać kody z różnych powierzchni.
Jak wygląda zarządzanie kodami QR?	Zarządzanie kodami QR w systemie obejmuje generowanie, weryfikację i unieważnianie kodów przez panel administracyjny, powiązane z profilem pracownika.	Administrator musi mieć możliwość zarządzania kodami.

Funkcjonalności (wymagania funkcjonalne)

Tabela 3: Opis funkcjonalności systemu SCANAR

Funkcjonalność	Opis	Uwagi
Dwuetapowa weryfikacja tożsamości	System wymaga najpierw zeskanowania kodu QR, a następnie weryfikacji twarzy za pomocą kamery. Oba etapy muszą się powieść, aby dostęp został przyznany.	Weryfikacja dwuetapowa znacząco zwiększa bezpieczeństwo w porównaniu z tradycyjnymi kartami dostępu.
Generowanie kodów QR	System automatycznie generuje unikalne kody QR dla każdego pracownika na podstawie jego ID. Kody są zapisywane jako pliki PNG 300x300px.	Kody QR mogą być drukowane na kartach dostępu lub wyświetlane na urządzeniach mobilnych.
Rozpoznawanie twarzy	Wykorzystanie biblioteki <code>face_recognition</code> do porównywania twarzy z kamery z wzorcami w bazie danych. Tolerancja dopasowania: 0.6 (im niższa, tym bardziej rygorystyczna weryfikacja).	System wymaga zdjęć twarzy pracownika dla dokładnej weryfikacji. Działa w czasie rzeczywistym.
Panel administracyjny (CRUD pracowników)	Interfejs webowy (React + Flask) umożliwiający: dodawanie nowych pracowników z danymi i zdjęciami, edycję danych istniejących pracowników, usuwanie pracowników, przeglądanie listy wszystkich pracowników, zarządzanie statusem (aktywny/nieaktywny).	Wymaga uwierzytelnienia.
Zarządzanie kodami QR	Administrator może: generować nowy kod QR dla pracownika, wyświetlać istniejący kod QR pracownika, pobierać kod QR jako plik PNG, dezaktywować kod QR (zmiana statusu na nieaktywny).	Dezaktywacja kodu QR natychmiast uniemożliwia dostęp bez usuwania danych pracownika z systemu.
Logowanie prób dostępu	Każda próba dostępu (udana i nieudana) jest rejestrowana z danymi: timestamp, ID pracownika, typ zdarzenia (INVALID_QR, FACE_MISMATCH,	5 typów zdarzeń umożliwia szczegółową analizę i wykrywanie prób oszustwa.

Funkcjonalność	Opis	Uwagi
	VERIFICATION_SUCCESS, QR_SCANNED, FACE_SCANNED), wynik podobieństwa twarzy (0-1), zdjęcie z próby.	
Statystyki i raporty	Automatyczne generowanie statystyk: liczba wszystkich prób dostępu, liczba udanych weryfikacji, liczba odrzuconych prób, rozkład po typach zdarzeń, najczęstsze godziny prób dostępu, lista pracowników z największą liczbą nieudanych prób.	Statystyki dostępne przez API i panel administracyjny, możliwość eksportu danych.
Uwierzytelnianie sesji	Panel administracyjny chroniony systemem sesji Flask. Sesje wygasają po zamknięciu przeglądarki.	W produkcji należy zmienić hasło administratora i używać silniejszych mechanizmów uwierzytelniania (np. OAuth).
Przechowywanie danych lokalnie	Baza danych SQLite (<code>instance/scanar.db</code>), zdjęcia twarzy w <code>/static/face_images/</code> , kody QR w <code>/static/qr_codes/</code> . Wszystkie dane przechowywane lokalnie na serwerze.	Brak zewnętrznych zależności, łatwe tworzenie backupów (kopiowanie folderów i bazy).
Dokumentacja API (Swagger)	Pełna dokumentacja REST API dostępna pod adresem <code>/apidocs/</code> z możliwością testowania endpointów bezpośrednio w przeglądarce.	Swagger UI generowany automatycznie z Flasgger, obsługa lokalnych plików statycznych.
Dokumentacja kodu (Sphinx)	Interaktywna dokumentacja funkcji	
Przewodnik uruchomienia aplikacji	Instrukcja w Setup.md w repozytorium	
Interfejs użytkownika (React)	Nowoczesny, responsywny interfejs webowy stworzony w React 19.0.2 z: podglądem kamery, przewodnikiem po procesie weryfikacji, listą pracowników z możliwością edycji, dodawaniem nowych	Vite 6.0.5 jako build tool, komunikacja z backendem przez REST API.

Funkcjonalność	Opis	Uwagi
	pracowników z przesyłaniem zdjęć, zarządzaniem kodami QR, przeglądaniem logów i statystyk. Ekran weryfikacji (widok główny): - Przycisk do rozpoczęcia procesu weryfikacji - Podgląd kamery w czasie rzeczywistym - Instrukcje dla użytkownika - Czytelny interfejs graficzny Proces skanowania kodu QR: - Dopasowanie przechwytywanego kodu do pola skanowania - Automatyczne wykrycie kodu QR - Alert sukcesu lub komunikat o błędzie (SweetAlert2) - Automatyczne przejście do weryfikacji twarzy Proces weryfikacji twarzy: - Automatyczne rozpoczęcie po skanowaniu QR - Dopasowanie twarzy do ramki pomocniczej - Wyświetlenie wyniku: "Dostęp przyznany" lub "Dostęp odrzucony" - Komunikat o powodzie odrzucenia (jeśli dotyczy) Widok po weryfikacji: - Komunikat podsumowujący (sukces/porażka) - Automatyczne przejście do widoku głównego po 3 sekundach - Przycisk do ponownej próby weryfikacji	

Funkcjonalność	Opis	Uwagi
	- Blokada czasowa po przekroczeniu liczby dozwolonych prób Panel administratora: - Okienko logowania/wylogowania - Dodawanie nowych pracowników (imię i nazwisko, stanowisko, dział) - Generowanie kodów QR dla pracowników - Upload do 5 zdjęć twarzy na pracownika - Zarządzanie pracownikami: - Edycja danych pracownika - Usuwanie pracownika z bazy - Aktywacja/dezaktywacja pracownika - Podgląd szczegółowych danych - Pobieranie kodu QR jako PNG - Przeglądanie logów zdarzeń: - Filtrowanie po typie zdarzenia, dacie, osobie - Wyświetlanie timestampów - Podgląd zdjęć z prób weryfikacji - Generowanie raportów PDF ze statystykami - Responsywny design działający na różnych urządzeniach	

Wymagania niefunkcjonalne

- **Wydajność:** Czas weryfikacji kodu qr i face recognition <3 sekundy
- **Dokładność:** ≥95% skuteczności rozpoznawania twarzy
- **Tolerancja:** 0.6 dla algorytmu face_recognition
- **Sprzęt:** Kamera minimum 720p, oświetlenie ≥300 lux, 4GB RAM
- **Bezpieczeństwo:** Uwierzytelnianie administratora, utrzymanie sesji
- **Dostępność:** System działa offline (lokalna baza SQLite)
- **Użyteczność:** Responsywny interfejs, komunikaty (wyskakujące okna)
- **Skalowalność:** Do 5 zdjęć na pracownika, lokalna baza
- **Zgodność:** RODO - możliwość usunięcia wszystkich danych pracownika

Ustalony format danych wejściowych

Dane wejściowe do systemu SCANAR:

1. **Zdjęcia twarzy pracownika** – format JPG, rozdzielczość minimum 640x480px, zalecane 1280x720px lub wyższe
2. **Obraz z kamery na żywo** – rozdzielczość kamery minimum 720p
3. **Kod QR** – obraz kodu QR skanowany przez kamerę, format PNG wygenerowany przez bibliotekę qrcode
4. **Dane osobowe pracownika** – imię, nazwisko, department, stanowisko

Tabela 4: Format danych w bazie

Tabela: Employee	Tabela: EventLog	Obsługiwane formaty plików
id (int, primary key, autoincrement)	id (int, primary key, autoincrement)	Zdjęcia: .jpg
name (string(100), wymagane)	event_type (string(50): INVALID_QR, FACE_MISMATCH, VERIFICATION_SUCCESS, QR_SCANNED, FACE_SCANNED)	Kody QR: .png
position (string(100), stanowisko)	employee_id (int, foreign key, nullable)	Baza danych: .db (SQLite)

Tabela: Employee	Tabela: EventLog	Obsługiwane formaty plików
department (string(100), dział)	qr_code_hash (string(255), hash kodu QR, nullable)	API: JSON
qr_code_hash (string(255), unique, hash QR)	image_path (string(500), ścieżka do zdjęcia z weryfikacji, nullable)	
face_encoding_1 - face_encoding_5 (binary, numpy arrays)	image_data (binary, dane obrazu, nullable)	
face_image_path_1 - face_image_path_5 (string(255), ścieżki do 5 zdjęć)	message (text, opis zdarzenia/błędu, nullable)	
is_active (boolean, default: True)	timestamp (datetime UTC, czas zdarzenia)	
created_at (datetime, default: CURRENT_TIMESTAMP)	created_at (datetime UTC, czas zapisu)	

Modelowanie systemu

Tabela 5: Modelowanie systemu za pomocą tabeli

Element	Opis
Aktorzy	Pracownik – osoba korzystająca z systemu weryfikacji dostępu Administrator Systemu – osoba zarządzająca systemem przez panel webowy
Opis	Celem systemu SCANAR jest weryfikacja tożsamości pracowników wchodzących do zakładu przemysłowego poprzez dwuetapowy proces: (1) skanowanie unikalnego kodu QR przypisanego do

Element	Opis
	pracownika, (2) weryfikacja twarzy za pomocą rozpoznawania biometrycznego. System ma zapobiegać nadużyciom związanym z przekazywaniem kart dostępowych (QR kodów) między osobami poprzez wymaganie fizycznej obecności uprawnionej osoby. System generuje szczegółowe logi dostępu i raporty dla celów audytu i bezpieczeństwa.
Dane	Baza danych SQLite: zawiera rekordy pracowników (ID, imię i nazwisko, stanowisko, dział, hash kodu QR, zakodowane reprezentacje twarzy 1-5, ścieżki do zdjęć twarzy 1-5, status aktywności, data utworzenia) oraz logi zdarzeń (ID logu, typ zdarzenia, ID pracownika, hash kodu QR, ścieżka do obrazu, dane obrazu, komunikat, timestamp zdarzenia, timestamp utworzenia) Pliki: zdjęcia twarzy (.jpg), kody QR (.png), zdjęcia z prób weryfikacji (.jpg) Parametry: tolerancja rozpoznawania twarzy (0.6), SECRET_KEY dla sesji, dane logowania administratora
Wyzwalacz	Proces weryfikacji: 1. Pracownik podchodzi do stanowiska kontroli dostępu wyposażonego w kamerę i wyświetlacz 2. Pracownik prezentuje kod QR (z telefonu lub kartki) przed kamerą 3. System skanuje i dekoduje kod QR → pobiera ID pracownika 4. System automatycznie przechodzi do etapu weryfikacji twarzy 5. System przechwytyuje obraz z kamery i porównuje z wzorcami w bazie Panel administracyjny:

Element	Opis
	Administrator loguje się do panelu i wykonuje operacje CRUD na pracownikach lub przegląda logi
Odpowiedź	Weryfikacja udana: System wyświetla komunikat "Dostęp przyznany", loguje zdarzenie VERIFICATION_SUCCESS. Weryfikacja nieudana: System wyświetla przyczynę odrzucenia (np. "Kod QR nieważny", "Twarz nie rozpoznana"), loguje odpowiednie zdarzenie (INVALID_QR lub FACE_MISMATCH), zapisuje zdjęcie z próby dla administratora Panel administracyjny: Administrator otrzymuje potwierdzenie wykonanej operacji (np. dodanie pracownika, wygenerowanie kodu QR) oraz dostęp do statystyk i logów

Diagram przypadków użycia

Rysunek 1: Diagram przypadków użycia dla aktorów systemu

Diagram przypadków użycia - System SCANAR

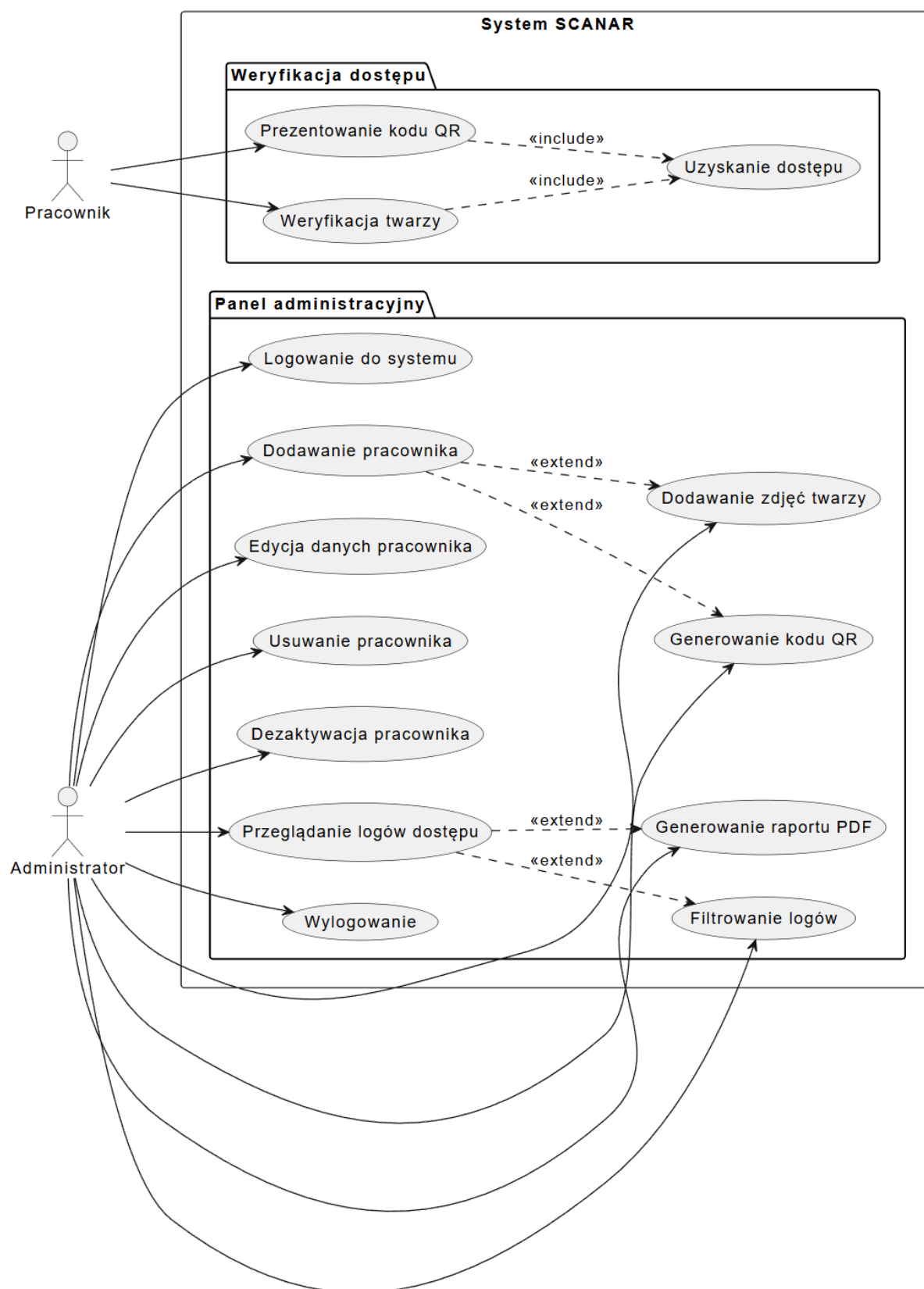
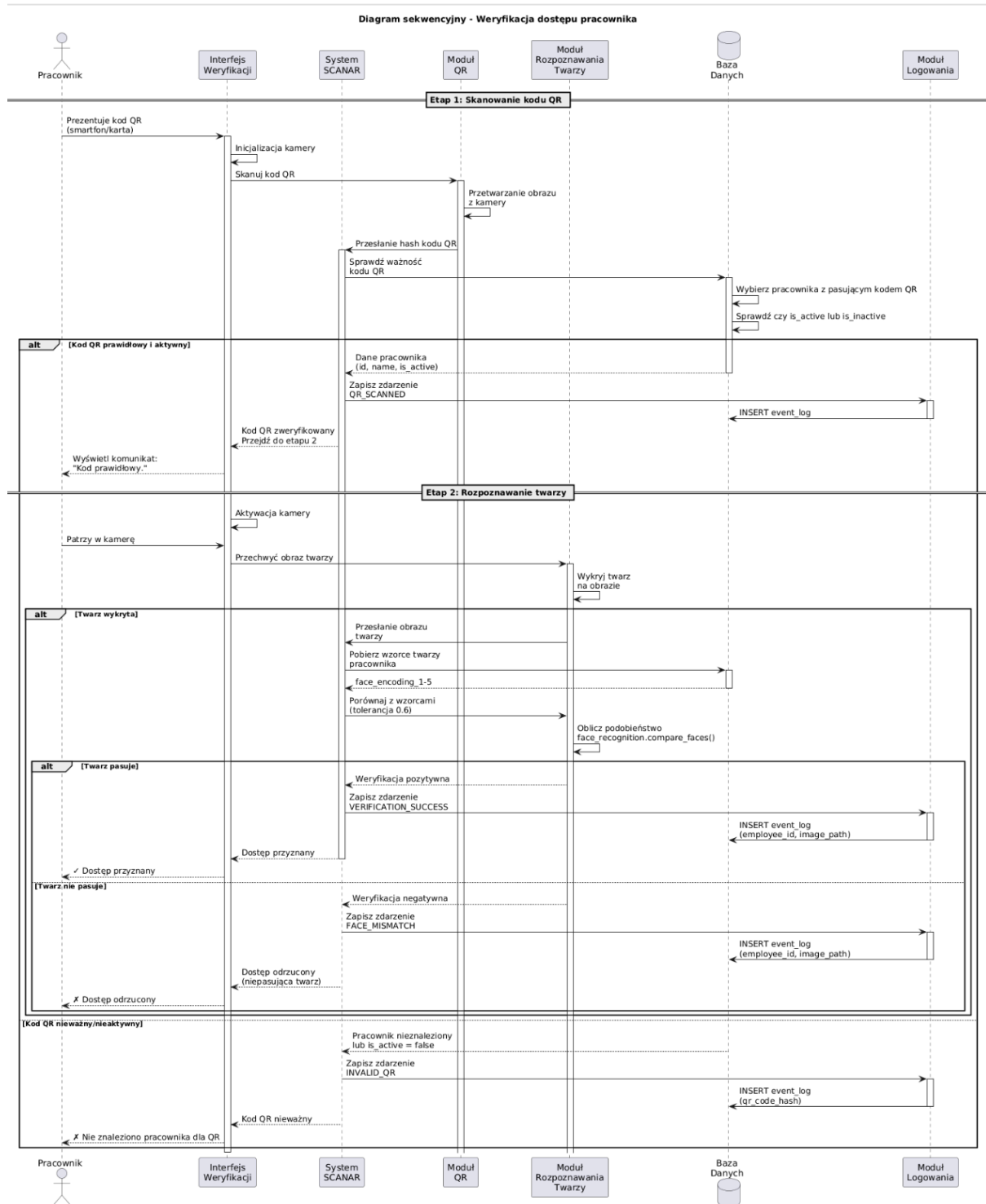
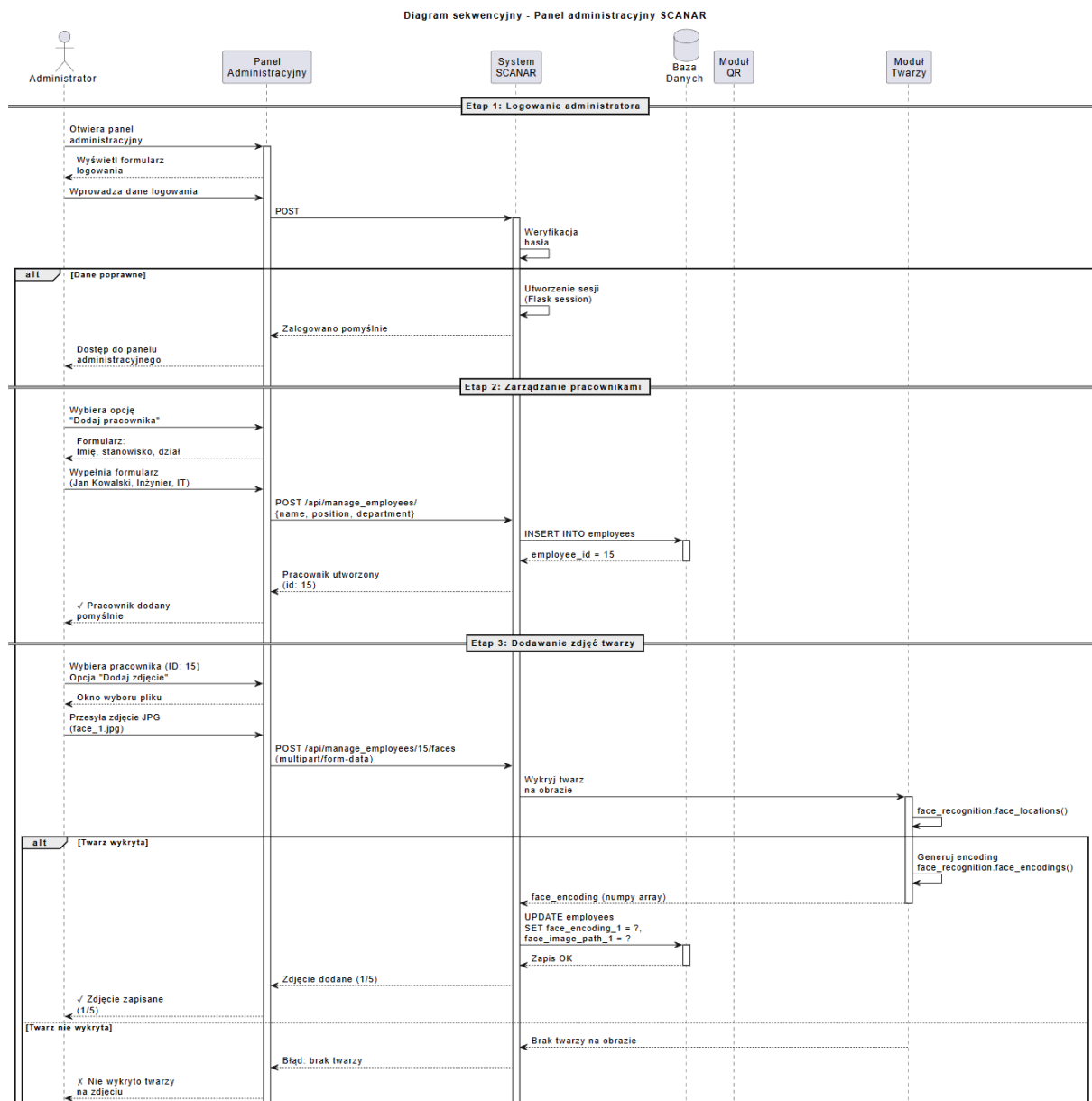


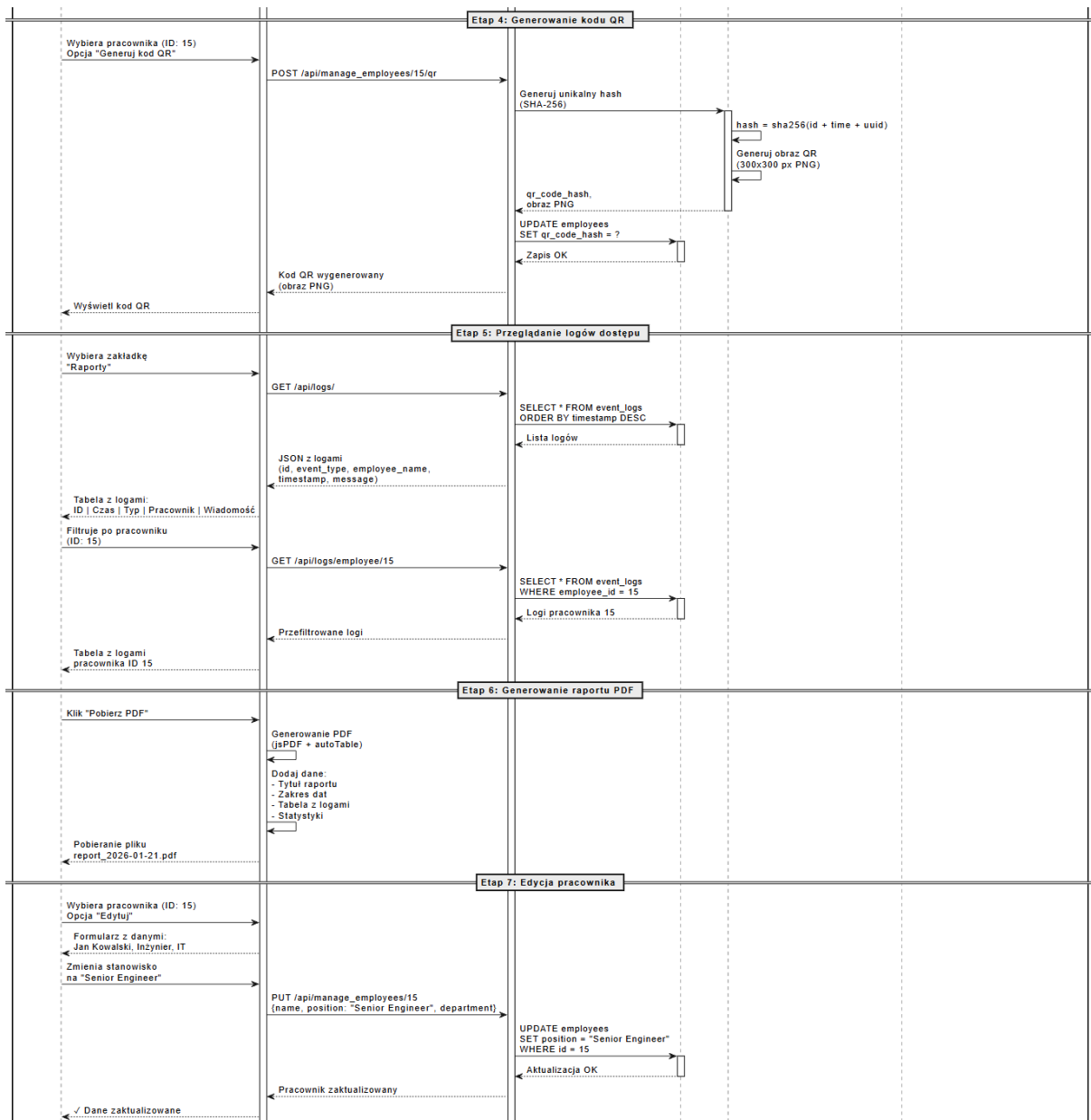
Diagram sekwencyjny UML

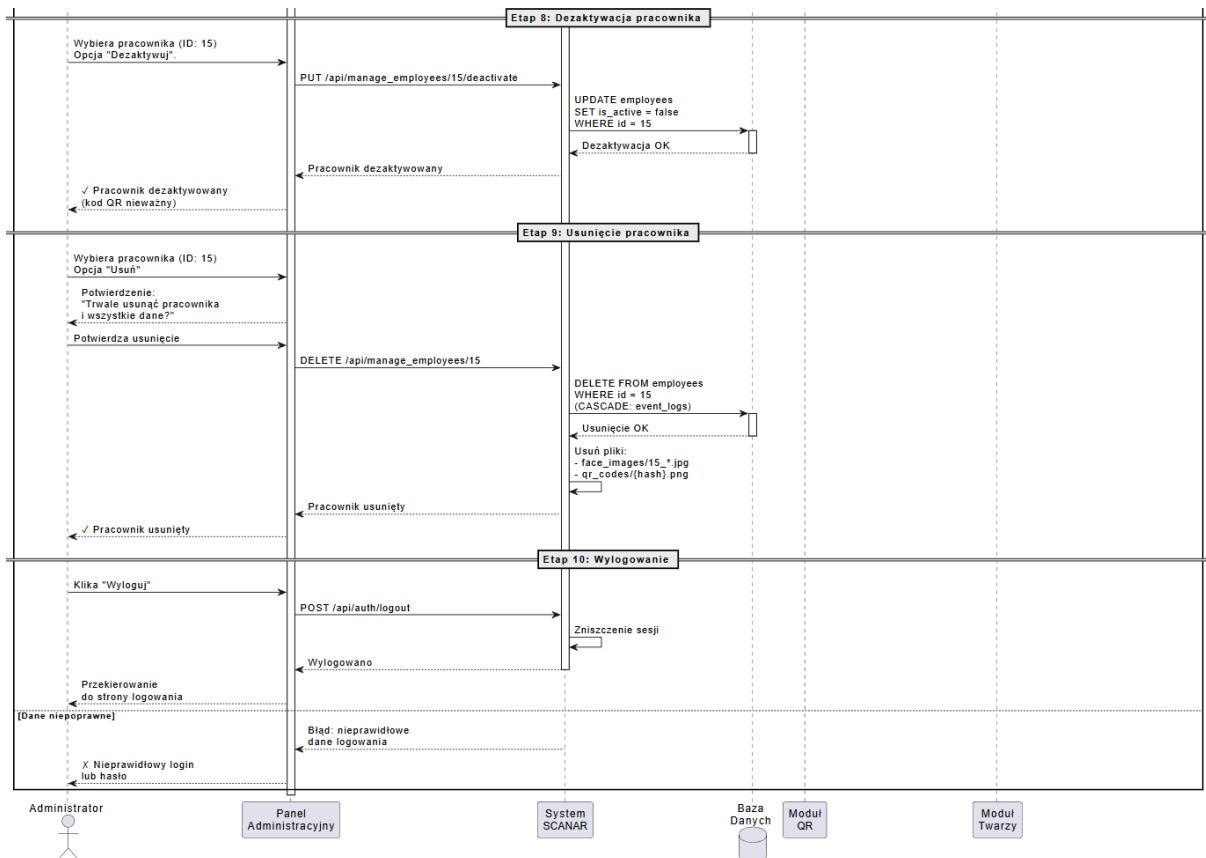
Pracownik (użytkownik)



Administrator







Projekt architektury opracowywanego systemu

- **Komponenty wejściowe:**
 - Skaner kodów QR.
 - Kamera wideo/aparat.
- **Komponenty przetwarzające:**
 - **Moduł Weryfikacji Kodu QR:** Odpowiedzialny za dekodowanie i sprawdzanie ważności kodu w bazie danych.
 - **Moduł Rozpoznawania Twarzy:** Kluczowy komponent, który analizuje obraz, wykrywa twarz i porównuje ją z zapisanymi wzorcami.
 - **Baza Danych:** Przechowuje dane pracowników, ich zdjęcia/wzorce biometryczne, uprawnienia i historię logowań.
 - **Panel Administracyjny:** Interfejs webowy dla administratorów do zarządzania systemem.

- **Komponenty wyjściowe:**
 - **Mechanizm sygnalizacyjny:** Okno informujące o statusie weryfikacji.
 - **Moduł Logowania:** Zapisuje każdą próbę dostępu do bazy danych.
 - **Moduł Raportowania:** Generuje raporty na żądanie administratora.

DIWMVP - Definicja Minimum Viable Product

Opis ogólny MVP:

MVP (Minimum Viable Product) dla systemu SCANAR obejmuje najbardziej podstawowe funkcjonalności niezbędne do przeprowadzenia dwuetapowej weryfikacji tożsamości pracowników (kod QR + rozpoznawanie twarzy) oraz zarządzania danymi pracowników przez administratora. System MVP jest zaprojektowany jako aplikacja lokalna z interfejsem webowym, wykorzystująca sprawdzone technologie open-source.

Podstawowe funkcjonalności MVP:

Funkcjonalność	Opis	Technologia
Dwuetapowa weryfikacja dostępu	Pracownik skanuje kod QR, następnie system weryfikuje jego twarz	Flask (backend), React (frontend), face_recognition
Zarządzanie pracownikami (CRUD)	Administrator może dodawać, edytować, dezaktywować i usuwać pracowników	Flask API, SQLite, React
Generowanie kodów QR	System automatycznie generuje unikalne kody QR dla każdego pracownika	qrcode library, Python
Upload zdjęć twarzy	Administrator może dodać do 5 zdjęć twarzy każdego pracownika do systemu	Flask file upload, SQLite
Logowanie zdarzeń dostępu	Każda próba dostępu (udana/nieudana) jest zapisywana w bazie danych z timestampem	EventLog table, SQLite

Funkcjonalność	Opis	Technologia
Podstawowy reporting	Administrator może przeglądać historię dostępów (udanych i nieudanych)	Flask API, React interface
Baza danych SQLite	Lokalna baza danych przechowująca dane pracowników i logi dostępu	SQLite (Employee, EventLog tables)
Interfejs React	Responsywny interfejs webowy dla weryfikacji dostępu i panelu administratora	React 19.0.2, Vite 6.0.5

Kryteria sukcesu MVP:

Kryterium	Wartość docelowa	Metoda weryfikacji
Dokładność rozpoznawania twarzy	$\geq 95\%$ przy tolerance=0.6	Testy z bazą zdjęć pracowników
Czas weryfikacji dwuetapowej	<3 sekundy (QR + twarz)	Pomiar czasu od skanowania QR do decyzji
Wskaźnik generowania kodów QR	100% (brak błędów generowania)	Automatyczne testy wszystkich kodów
Zero nieautoryzowanych dostępów	100% odrzucenie osób nieznanych	Testy z nieznanymi twarzami
Kompletność logowania	100% zdarzeń zapisanych z timestampem	Audit bazy EventLog

Wymagania sprzętowe MVP:

- PC/Laptop: min 4GB RAM
- Kamera: rozdzielczość minimum 720p
- Oświetlenie: minimum 300 lux w miejscu weryfikacji
- Przeglądarka: Chrome/Firefox w najnowszej wersji

Dokumentacja techniczna MVP:

- Code Documentation (Sphinx)
- Dokumentacja API (Swagger)

- Instrukcja instalacji i konfiguracji
- Dokumentacja testów (osobny dokument)
- README z opisem architektury i technologii

System musi być zaprojektowany w sposób modułowy. Krytycznym elementem jest algorytm rozpoznawania twarzy, który musi być zarówno szybki (aby nie tworzyć kolejek przy wejściu), jak i dokładny (aby zminimalizować liczbę fałszywych odrzuceń i akceptacji). Skuteczność tego modułu bezpośrednio wpływa na wiarygodność całego systemu i generowanych raportów.

Stos technologiczny

Backend:

Technologia	Wersja	Przeznaczenie
Python	3.11	Język programowania backendu
Flask	3.0.0	Framework webowy do budowy REST API
Flask-CORS	4.0.0	Obsługa Cross-Origin Resource Sharing
Flask-SQLAlchemy	3.1.1	ORM do obsługi bazy danych SQLite
Flasgger	0.9.7.1	Automatyczna dokumentacja API (Swagger/OpenAPI)
face_recognition	1.3.0	Biblioteka do rozpoznawania twarzy (oparta na dlib)
NumPy	1.24.0	Operacje na tablicach i przetwarzanie danych biometrycznych
Pillow	≥9.5.0	Przetwarzanie i manipulacja obrazami

Technologia	Wersja	Przeznaczenie
python-dotenv	1.0.0	Zarządzanie zmiennymi środowiskowymi
SQLite	3.x	Relacyjna baza danych (wbudowana w Python)

Frontend:

Technologia	Wersja	Przeznaczenie
React	19.2.0	Biblioteka JavaScript do budowy interfejsu użytkownika
Vite	7.2.4	Narzędzie do budowania i bundlowania aplikacji
React Router DOM	7.12.0	Routing i nawigacja w aplikacji SPA
html5-qrcode	2.3.8	Skanowanie kodów QR przez przeglądarkę (WebRTC)
qrcode.react	4.2.0	Generowanie i wyświetlanie kodów QR w React
SweetAlert2	11.26.17	Stylowe alerty i okna dialogowe
jsPDF	4.0.0	Generowanie dokumentów PDF
jspdf-autotable	5.0.7	Tworzenie tabel w dokumentach PDF
ESLint	9.39.1	Linter JavaScript do kontroli jakości kodu

Uzasadnienie wyboru technologii:

- **Flask** został wybrany ze względu na prostotę, elastyczność i bogaty ekosystem rozszerzeń, idealny dla aplikacji REST API o średniej złożoności
- **face_recognition** opiera się na bibliotece dlib i oferuje wysoką dokładność rozpoznawania twarzy przy relatywnie prostym API

- **SQLite** zapewnia lokalną, bezserwerową bazę danych, eliminując potrzebę zewnętrznej infrastruktury bazy danych
- **React 19** z Vite zapewnia nowoczesny, wydajny frontend z szybkim czasem budowania i hot module replacement
- **html5-qrcode** umożliwia skanowanie QR bezpośrednio w przeglądarce bez dodatkowych zależności natywnych
- **SweetAlert2** poprawia user experience poprzez atrakcyjne wizualnie komunikaty i alerty

Informacje

Wersja dokumentu: 1.0

Autorzy: Paweł Kowalcze, Mateusz Gacek, Jan Ogiegło